

Full-Stack Architecture Specification

Prepared by: Ajinkya Moharir | Project: FullStack Intern Coding Challenge

1. Technology Stack

- **Frontend:** React.js
- **Backend:** Express.js (Node.js)
- **Database:** PostgreSQL

2. Database Schema

Users Table (Single Login System)

Column	Data Type	Constraints / Notes
id	UUID	Primary Key
name	VARCHAR(60)	Min 20, Max 60 chars
email	VARCHAR(255)	Unique, Valid Email
password_hash	VARCHAR(255)	Bcrypt Hash (8-16 chars plain)
address	VARCHAR(400)	Max 400 chars
role	ENUM	'SYSTEM_ADMIN', 'NORMAL_USER', 'STORE_OWNER'
created_at	TIMESTAMP	Default Current Time

Stores Table

Column	Data Type	Constraints / Notes
id	UUID	Primary Key
owner_id	UUID	Foreign Key -> Users(id)

name	VARCHAR(255)	-
email	VARCHAR(255)	-
address	VARCHAR(400)	-
created_at	TIMESTAMP	Default Current Time

Ratings Table (Junction Table)

Column	Data Type	Constraints / Notes
id	UUID	Primary Key
user_id	UUID	Foreign Key -> Users(id)
store_id	UUID	Foreign Key -> Stores(id)
score	INTEGER	Check: 1 to 5
created_at	TIMESTAMP	Default Current Time

Constraint: Unique compound index on (user_id, store_id) to prevent duplicate ratings.

3. API Endpoints (RESTful)

All endpoints return JSON. Protected endpoints require a valid JWT.

Authentication Endpoints

- **POST /api/auth/signup** - Registers a Normal User.
- **POST /api/auth/login** - Authenticates user and returns JWT.
- **PUT /api/auth/password** - Updates user password (Requires JWT).
- **POST /api/auth/logout** - Invalidates session/token.

Admin Endpoints (Requires 'SYSTEM_ADMIN' Role)

- **GET /api/admin/dashboard** - Returns aggregate counts: total users, stores, and ratings.
- **POST /api/admin/users** - Creates an Admin or Normal User.

- **GET /api/admin/users** - Retrieves user list. Supports query parameters for filters/sorting (e.g., `?role=NORMAL_USER&sort=name:asc`).
- **POST /api/admin/stores** - Creates a store and assigns to an owner.

Store & Rating Endpoints

- **GET /api/stores** - Retrieves stores with dynamically calculated average ratings. Supports query params (e.g., `?search=xyz&address=abc`).
- **POST /api/stores/:storeId/ratings** - Submits or updates a 1-5 rating for a specific store (Requires 'NORMAL_USER' Role).

Store Owner Endpoints (Requires 'STORE_OWNER' Role)

- **GET /api/owner/dashboard** - Returns the specific store's average rating and list of users who rated it.

4. Frontend Architecture

- **Routing & Security:** Utilizes a Higher-Order Component `<ProtectedRoute />` wrapper using `react-router-dom` to intercept unauthorized access based on role.
- **State Management:** Employs React **Context API** (`AuthContext`) for lightweight, global access to the active user's JWT and profile data.
- **UI Reusability:** A centralized `<DataTable />` component handling data, columns, `onSort`, and `enableFilters` props to ensure consistent sorting/filtering functionality across Admin, Store Owner, and Normal User views.